

Refactoring — Group / Ungroup figures (JHotDraw 7)

Target class: `org.jhotdraw.draw.action.GroupAction` **Branch:** `refactor/group-ungroup-compose-method` (feature-branch / GitHub flow) **Smell source:** Chapter 4 "Code Smells", *Refactoring to Patterns* — Kerievsky 2005 [Ker05]

1. The code smell that triggered the refactoring

`GroupAction.actionPerformed(...)` was a **69-line method** exhibiting three of the smells catalogued in Chapter 4 of [Ker05]:

- **Long Method** — the whole grouping *and* ungrouping algorithms (selection handling, prototype cloning, building an `UndoableEdit`, mutating the drawing, firing the edit) were inlined in one method.
- **Duplicated Code** — the two branches were near-mirror images: each built an anonymous `AbstractUndoableEdit` with the same `redo` / `undo` scaffolding, and the resource-bundle lookup `DrawLabels.getLabels().getString(...)` was written out **twice**.
- **Conditional Complexity** — a boolean control flag, `isGroupingAction`, selects between the two behaviours (`if (isGroupingAction) ... else ...`), so a single method does two different jobs.

SonarLint (installed for this lab) reports the matching rules on this method: `java:S3776` (Cognitive Complexity too high), `java:S1192` (duplicated string literal / repeated logic) and `java:S1188` (overly long anonymous classes).

2. What I planned to change

Restructure `actionPerformed` so it reads at a single level of abstraction and the grouping and ungrouping responsibilities are separated, **without changing any external behaviour** (same selection effects, same undoable edits, same labels).

3. Strategy of the refactorings

Work in small, behaviour-preserving steps on a feature branch:

1. **Extract Method** the repeated resource-bundle lookup into `labelText(key)`.
2. **Extract Method** each branch into `groupSelectedFigures()` and `ungroupSelectedFigures()`, moving the `can...()` guard and the anonymous edit into the extracted method.
3. **Compose Method** the now-tiny `actionPerformed` so it only dispatches: `if (isGroupingAction) groupSelectedFigures(); else ungroupSelectedFigures();`.

4. Refactorings applied (from [Ker05] / Fowler catalog referenced therein)

Refactoring	Where	Reasoning
Compose Method (Kerievsky, p.123)	<code>actionPerformed</code>	Turns the long method into a short, intention-revealing one whose body operates at one level of abstraction — the primary cure for the Long Method smell.
Extract Method (Fowler)	<code>groupSelectedFigures()</code> , <code>ungroupSelectedFigures()</code> , <code>labelText()</code>	Gives each responsibility a name, isolates the two algorithms, and removes the duplicated bundle lookup (cure for Duplicated Code).

Strategic next step (described, not yet applied)

The remaining **Conditional Complexity** is the `isGroupingAction` flag. The catalogued cure is **Replace Conditional with Polymorphism** (Fowler) — Kerievsky's *Replace Conditional Logic with Strategy*: drop the flag and let `GroupAction` perform grouping while `UngroupAction` overrides the operation. This was *not* applied here because the 3-argument constructor carrying the flag is public API used by menu factories, so removing it is a larger, separate step that would ripple to callers. It is recorded as the next refactoring in the sequence.

5. Before / after (evidence)

Before — one 69-line method, two duplicated branches:

```
public void actionPerformed(ActionEvent e) {
    if (isGroupingAction) {
        if (canGroup()) {
            ... 30 lines: clone prototype, anonymous UndoableEdit, groupFigures, fire ...
        }
    } else {
        if (canUngroup()) {
            ... 30 lines: mirror image with ungroupFigures ...
        }
    }
}
```

After — Compose Method + Extract Method:

```
public void actionPerformed(ActionEvent e) {
    if (isGroupingAction) {
        groupSelectedFigures();
    }
}
```

```
    } else {  
        ungroupSelectedFigures();  
    }  
}  
  
private void groupSelectedFigures() { ... }    // guard + edit, one responsibility  
private void ungroupSelectedFigures() { ... }  // guard + edit, one responsibility  
private static String labelText(String key) {  // removes the duplicated lookup  
    return DrawLabels.getLabels().getString(key);  
}
```

6. Behaviour preservation

The transformation is purely structural: the same `groupFigures` / `ungroupFigures` calls run in the same order, the same `AbstractUndoableEdit` objects are fired with the same presentation labels, and the redundant no-op `addEdit` override was dropped (its default behaviour is unchanged). No public method signature changed, so the external behaviour of the action is identical — the heart of a refactoring per the lab's definition.